

Infrastructure Automation with Ansible and Kernel Auditing via eBPF (Tetragon)

2026

Author: Rodríguez Rubén

INDICE:

INTRODUCTION:	3
Hardware Resource Allocation	4
Step 1. Environment initialization & Connectivity Control	5
1.1 Network Topology & IP Assignment	5
1.2 SSH Key-Based Authentication	5
1.3 Ansible Control Plane & Inventory Configuration	6
Step 2. Infrastructure as Code (IaC) — Automation with Ansible	7
2.1 Workspace Directory Architecture & Blueprint	7
2.2 Declarative Playbook Engineering (deploy-lab.yml)	8
2.3 Provisioning Execution & Deployment Controls	9
Step 3. Security Agent Deployment & Target Workload Isolation	9
3.1 Kernel Observability Architecture (Cilium Tetragon)	10
3.2 Target Vulnerable Environment Definition	11
3.3 Security Agent Deployment & Target Workload Isolation	12
Step 4. Runtime Auditing & Attack Simulation	12
4.1 Attack Vector: Container Shell Intrusion (RCE Simulation)	12
4.1.1 Declarative Volume Mapping & Container Identification (Playbook Optimization)	13
4.1.2 Exploit Execution via Absolute Node.js Path	14
Step 5. Static Vulnerability Analysis with Trivy	15
5.1 Image Vulnerability Scanning & Software Composition Analysis (SCA)	15
5.1.2 Static Metric Assessment & Vulnerability Vectors	17

INTRODUCTION:

Containerized application deployment requires a shift in modern defense strategies: protecting the network perimeter is no longer enough; monitoring internal system behavior in real-time is now critical.

This project implements an advanced security laboratory under the DevSecOps methodology, structured into three core pillars:

- **Automation:** Deployment of the base infrastructure using Ansible as Infrastructure as Code (IaC).
- **Prevention (*Shift Left*):** Static vulnerability analysis in Docker images utilizing Trivy.
- **Runtime Detection:** Invisible auditing of system calls (*syscalls*) directly inside the Linux Kernel using eBPF technology (Tetragon).

Step 0. Preparation.

Before deploying the laboratory, it is essential to outline the technical prerequisites and resource allocations. The entire environment is virtualized utilizing a Type-2 Hypervisor, specifically **VMware Workstation Pro**.

To ensure optimal performance while running the target applications alongside kernel-level security monitoring tools, the host machine and the target Virtual Machine (VM) must meet specific hardware specifications.

Hardware Resource Allocation

- **Host Machine Requirements:** A robust multi-core processor (e.g., AMD Ryzen 5/7 or Intel i5/i7) and a minimum of 16 GB of RAM (32 GB recommended) to handle virtualization overhead smoothly.

Resource	Client VM (Management / Attacker)	Target Server VM (Production)
OS	Kali Linux / Ubuntu Desktop	Ubuntu Server (Latest version)
vCPU	2 Cores	2 to 4 Cores
RAM	2 GB to 4 GB	4 GB to 8 GB
Storage	45 GB (SSD)	45 GB (SSD)
Kernel	Standard	Linux Kernel 5.15+ (Mandatory)
Network	1 vNIC (NAT / Host-Only)	1 vNIC (NAT / Host-Only)

IMPORTANT: This documentation focuses exclusively on the technical implementation of the project and does not cover the standard virtual machine creation wizard procedure.

Step 1. Environment initialization & Connectivity Control

1.1 Network Topology & IP Assignment

To facilitate seamless automation and vulnerability testing, a static network configuration has been established within the VMware private virtual network. This ensures that the management node can consistently reach the production target without relying on dynamic lease changes.

- **Management Node (Client):** Configured via DHCP/Static within the same subnet (e.g., 192.168.X.X).
- **Target Server:** Manually assigned a static IP address during the Ubuntu Server installation phase to ensure persistent SSH routing.

1.2 SSH Key-Based Authentication.

To allow Ansible to orchestrate the infrastructure securely and without interactive password prompts, an SSH key pair must be generated on the Management Node and deployed to the Target Server. Using asymmetric cryptography (ED25519) ensures a secure, automated connection.

Firstly, generate the SSH Key Pair (on the client).

```
ssh-keygen -t ed25519 -C "ansible-management"
```

```
usuario@usuario-VMware-Virtual-Platform:~$ ssh-keygen -t ed25519 -C "ansible-manageme
nt"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/usuario/.ssh/id_ed25519):
Enter passphrase for "/home/usuario/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/usuario/.ssh/id_ed25519
Your public key has been saved in /home/usuario/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:vns/yTRiKwEIZ30Ls9fnwQI+/2sTtrxxSoAKyx+jXVo ansible-management
The key's randomart image is:
+--[ED25519 256]--+
|
| . + .
| + = .
| + + .
| . +.+S.
| . o..+..0 o
| o +.Eo.Oo0.o
| + * oo0++
| . + o*==..
+-----[SHA256]-----+
```

Transfer the Public key to the Target Server:

```
usuario@usuario-VMware-Virtual-Platform:~$ ssh-copy-id -i ~/.ssh/id_ed25519.pub ruben@192.168.111.20
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/usuario/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
ruben@192.168.111.20's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i /home/usuario/.ssh/id_ed25519 'ruben@192.168.111.20'"
and check to make sure that only the key(s) you wanted were added.

usuario@usuario-VMware-Virtual-Platform:~$
```

Verify Passwordless Access.

```
usuario@usuario-VMware-Virtual-Platform:~$ ssh ruben@192.168.111.20
Enter passphrase for key '/home/usuario/.ssh/id_ed25519':
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-27-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Jul 7 03:44:32 PM UTC 2026

System load: 0.0          Processes:                235
Usage of /:  34.1% of 20.99GB Users logged in:             0
Memory usage: 5%          IPv4 address for ens33: 192.168.111.20
Swap usage:  0%
```

1.3 Ansible Control Plane & Inventory Configuration

The infrastructure is mapped inside Ansible using a declarative inventory file (inventory.ini). This file explicitly groups the production assets and defines the necessary variables for remote connection management.

Create the inventory.ini file within your project's working directory on the client machine

```
GNU nano 8.7.1 inventory.ini
[production_servers]
target_server ansible_host=192.168.111.20 ansible_user=ruben ansible_ssh_private_key_file=~/.ssh/id_ed25519

[all:vars]
ansible_python_interpreter=/usr/bin/python3
```

Important, it is very necessary to execute the first command, because with that command we start the agent of ansible and the second, it is for adding the key in our client.

```
usuario@usuario-VMware-Virtual-Platform:~/server$ eval "$(ssh-agent -s)"
Agent pid 19223
usuario@usuario-VMware-Virtual-Platform:~/server$ ssh-add ~/.ssh/id_ed25519
```

For check if we have connection with the server, we use the following command.

ansible all -m ping -i inventory.ini

```
usuario@usuario-VMware-Virtual-Platform:~/server$ ansible all -m ping -i inventory.ini
target_server | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
usuario@usuario-VMware-Virtual-Platform:~/server$ ^C
usuario@usuario-VMware-Virtual-Platform:~/server$
```

```
Last login: Tue Jul 17 21:55 2020 from 192.168.111.199
ruben@kernel:~$ echo "ruben ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/ruben
[sudo: authenticate] Password:
ruben ALL=(ALL) NOPASSWD:ALL
ruben@kernel:~$
```

Step 2. Infrastructure as Code (IaC) — Automation with Ansible

2.1 Workspace Directory Architecture & Blueprint

To adhere to DevSecOps best practices, the deployment workspace on the Management Node is structured configurationally to isolate host inventories from orchestration playbooks.

2.2 Declarative Playbook Engineering (deploy-lab.yml)

This subsection details the automated, idempotent configuration of the Target Server. The playbook orchestrates full system package modernization, dependency management, secure GPG keyring isolation, and the deployment of the official stable Docker containerization engine without requiring human intervention.

Part of the deploy.yml, full version, in my github.

```
usuario@usuario-VMware-Virtual-Platform:~/server/ebpf-security$ cat deploy-lab.yml
---
- name: Provision Target Server and Install Docker Infrastructure
  hosts: production_servers
  become: true
  tasks:

    - name: Ensure system repositories are updated and upgraded
      apt:
        update_cache: yes
        upgrade: dist
        cache_valid_time: 3600

    - name: Install mandatory system dependencies
      apt:
        name:
          - ca-certificates
          - curl
        state: present

    - name: Create directory for Docker APT keyring
      file:
        path: /etc/apt/keyrings
        state: directory
        mode: '0755'
```

Here, we execute the ansible command to install all required of docker.

```
usuario@usuario-VMware-Virtual-Platform:~/server/ebpf-security$ ansible-playbook -i inventory.ini deploy-lab.yml

PLAY [Provision Target Server and Install Docker Infrastructure] *****
TASK [Gathering Facts] *****
ok: [target_server]

TASK [Ensure system repositories are updated and upgraded] *****
changed: [target_server]

TASK [Install mandatory system dependencies] *****
ok: [target_server]

TASK [Create directory for Docker APT keyring] *****
ok: [target_server]

TASK [Download Docker official GPG signing key (.asc)] *****
changed: [target_server]

TASK [Set up the stable Docker repository (DEB822 format)] *****
changed: [target_server]

TASK [Install official Docker CE Engine and CLI packages] *****
changed: [target_server]

TASK [Ensure Docker service is enabled and running] *****
ok: [target_server]

PLAY RECAP *****
target_server : ok=8 changed=4 unreachable=0 failed=0 skipped=0 rescued=0 ignore
d=0

usuario@usuario-VMware-Virtual-Platform:~/server/ebpf-security$
```


2.3 Provisioning Execution & Deployment Controls

This area covers the technical execution parameters required to invoke the automation control plane from the Management Node, migrating the Target Server into a fully functional production-ready container host.

To trigger the automated orchestration baseline, execute the following command:

```
ansible-playbook -i inventory.ini deploy-lab.yml
```

Post-Deployment Verification Controls:

```
sudo systemctl status docker --no-pager
```

```
sudo docker version
```

```
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-07-07 17:24:22 UTC; 10min ago
     Invocation: 2b6a955d300340789521f820068fd739
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 19057 (dockerd)
      Tasks: 11
     Memory: 30.4M (peak: 31.6M)
        CPU: 616ms
    CGroup: /system.slice/docker.service
            └─19057 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Jul 07 17:24:21 kernel dockerd[19057]: time="2026-07-07T17:24:21.784084152Z" level=info msg="Restoring _start."
Jul 07 17:24:21 kernel dockerd[19057]: time="2026-07-07T17:24:21.830147939Z" level=info msg="Deleting n...atus 1"
Jul 07 17:24:21 kernel dockerd[19057]: time="2026-07-07T17:24:21.845738201Z" level=info msg="Deleting n...atus 1"
Jul 07 17:24:22 kernel dockerd[19057]: time="2026-07-07T17:24:22.244661829Z" level=info msg="Loading co... done."
Jul 07 17:24:22 kernel dockerd[19057]: time="2026-07-07T17:24:22.255894817Z" level=info msg="Docker dae...=29.6.1
Jul 07 17:24:22 kernel dockerd[19057]: time="2026-07-07T17:24:22.256191724Z" level=info msg="Initializ...ildkit"
Jul 07 17:24:22 kernel dockerd[19057]: time="2026-07-07T17:24:22.353852478Z" level=info msg="Completed _...zation"
Jul 07 17:24:22 kernel dockerd[19057]: time="2026-07-07T17:24:22.361912348Z" level=info msg="Daemon has...zation"
Jul 07 17:24:22 kernel dockerd[19057]: time="2026-07-07T17:24:22.362201645Z" level=info msg="API listen...r.sock"
Jul 07 17:24:22 kernel systemd[1]: Started docker.service - Docker Application Container Engine.
Hint: Some lines were ellipsized, use -l to show in full.
Client: Docker Engine - Community
 Version: 29.6.1
  API version: 1.55
Go version: go1.26.4
 Git commit: 8900f1d
   Built: Fri Jun 26 11:40:38 2026
 OS/Arch: linux/amd64
 Context: default

Server: Docker Engine - Community
 Engine:
  Version: 29.6.1
  API version: 1.55 (minimum version 1.40)
Go version: go1.26.4
 Git commit: 8ec5ab3
   Built: Fri Jun 26 11:40:38 2026
 OS/Arch: linux/amd64
 Experimental: false
 containerd:
  Version: v2.2.5
  GitCommit: e53c7c1516c3b2bfff98eb76f1f4117477e6f4e66
 runc:
  Version: 1.3.6
  GitCommit: v1.3.6-0-g491b69ba
 docker-init:
  Version: 0.19.0
```

Step 3. Security Agent Deployment & Target Workload Isolation

This section details the orchestration and instantiation of both the monitoring control plane and the runtime target environments. To achieve full kernel-level observability, **Cilium Tetragon** is deployed as an eBPF-driven security agent capable of non-intrusive system call auditing. Concurrently, an isolated, intentionally vulnerable container environment is provisioned on the Target Server to serve as a secure sandbox for simulating and detecting real-world attack vectors.

3.1 Kernel Observability Architecture (Cilium Tetragon)

To interface directly with the Linux kernel without modifying its source code or injecting heavy monitoring modules, Cilium Tetragon is instantiated as a privileged container. This deployment profile permits the agent to load specific eBPF programs into the kernel space, monitoring process lifecycles, file integrity metadata, and network socket transitions directly from the underlying host execution layer.

The orchestration task requires full access to the host's PID namespace and the local Docker socket interface. The operational state is managed within our unified playbook via the following declarative structure:

```
- name: Deploy Cilium Tetragon eBPF Security Agent
  community.docker.docker_container:
    name: tetragon
    image: quay.io/cilium/tetragon:v1.2.0
    state: started
    restart_policy: always
    pid_mode: host
    privileged: true
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - /sys/kernel/debug:/sys/kernel/debug
```

```
usuario@usuario-VMware-Virtual-Platform:~/server/ebpf-security$ ansible-playbook -i inventory.ini deploy-lab.yml
PLAY [Provision Target Server and Install Docker Infrastructure] *****

TASK [Gathering Facts] *****
ok: [target_server]

TASK [Ensure system repositories are updated and upgraded] *****
ok: [target_server]

TASK [Install mandatory system dependencies] *****
ok: [target_server]

TASK [Create directory for Docker APT keyring] *****
ok: [target_server]

TASK [Download Docker official GPG signing key (.asc)] *****
ok: [target_server]

TASK [Set up the stable Docker repository (DEB822 format)] *****
ok: [target_server]

TASK [Install official Docker CE Engine and CLI packages] *****
ok: [target_server]

TASK [Ensure Docker service is enabled and running] *****
ok: [target_server]

TASK [Deploy Cilium Tetragon eBPF Security Agent] *****
changed: [target_server]

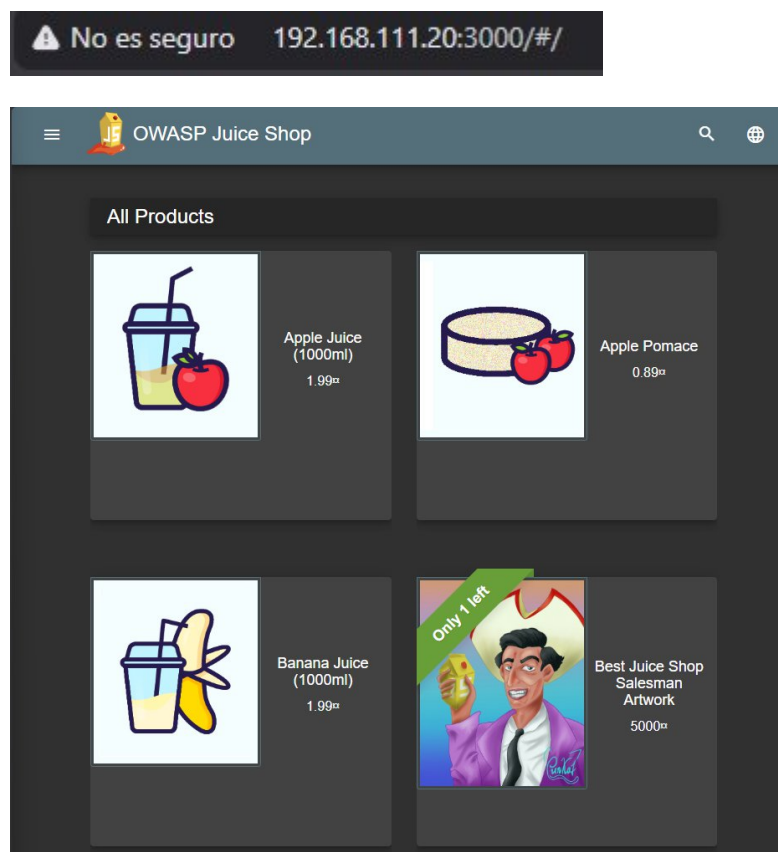
PLAY RECAP *****
target_server : ok=9 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignore=0
```

3.2 Target Vulnerable Environment Definition

To safely simulate modern cloud-native attack vectors (such as remote code execution or container escapes), an isolated, vulnerable microservice wrapper is provisioned alongside the monitoring tools. This target mimics a legacy unpatched environment, serving as the benchmark for security alerting validation.

```
- name: Deploy Intentionally Vulnerable Target Workload
  community.docker.docker_container:
    name: vulnerable-app
    image: bkimminich/juice-shop:v16.0.0
    state: started
    restart_policy: always
    ports:
      - "3000:3000"
```

Put the name server or IP in the bar browser, and then, we have the vulnerable app.



3.3 Security Agent Deployment & Target Workload Isolation

This section details the orchestration, instantiation, and runtime integration of both the security monitoring control plane and the target testing environment. To achieve absolute, kernel-level visibility without modifying the underlying operating system source code, **Cilium Tetragon** is provisioned as an eBPF-driven auditing daemon. Concurrently, an isolated, intentionally vulnerable application container is deployed on the Target Server to establish a secure, controlled sandbox designed for simulating, capturing, and validating real-world modern cloud-native exploit sequences.

In the same playbook which we have the deploy of the infrastructure, we add the container of tetragon.

```
- name: Deploy Cilium Tetragon eBPF Security Agent
  community.docker.docker_container:
    name: tetragon
    image: quay.io/cilium/tetragon:v1.2.0
    state: started
    restart_policy: always
    pid_mode: host
    privileged: true
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - /sys/kernel/debug:/sys/kernel/debug
```

```
ruben@kernel:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
01a4f3bc927a	bkimminich/juice-shop:v16.0.0	"/nodejs/bin/node /j..."	45 hours ago	Up 13 minutes	0.0.0.0
:3000->3000/tcp	vulnerable-app				
598f869ac990	quay.io/cilium/tetragon:v1.2.0	"/usr/bin/tetragon"	45 hours ago	Up 13 minutes	
	tetragon				

```
ruben@kernel:~$
```

Step 4. Runtime Auditing & Attack Simulation

This section demonstrates the active runtime detection phase of the DevSecOps laboratory. By executing a post-exploitation simulation inside the containerized infrastructure, the experiment validates how kernel-space instrumentation bypasses user-space limitations, capturing low-level execution data in real time.

4.1 Attack Vector: Container Shell Intrusion (RCE Simulation)

To simulate a critical post-exploitation event—such as a remote code execution (RCE) triumph or an administrative compromise—an interactive shell must be spawned inside the boundary of the unpatched microservice. In an enterprise breach scenario, an attacker attempts to establish an interactive foothold (`/bin/sh` or `/bin/bash`) to perform local reconnaissance, pivot through the internal network, or modify critical configuration files.

4.1.1 Declarative Volume Mapping & Container Identification (Playbook Optimization)

To ensure the security daemon tracks isolation spaces across distinct container runtimes, the declarative Ansible playbook configuration was updated to establish explicit persistence links. By mapping the underlying host Linux Control Groups (cgroups) interface, the kernel agent gains structural awareness of container boundaries:

The new change in the playbook, with this change, we can have the container of Tetragon connect with the root of containers.

```
- name: Deploy Cilium Tetragon eBPF Security Agent
  community.docker.docker_container:
    name: tetragon
    image: quay.io/cilium/tetragon:v1.2.0
    state: started
    restart_policy: always
    pid_mode: host
    privileged: true
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - /sys/fs/cgroup:/sys/fs/cgroup
      - /sys/kernel/btf/vmlinux:/sys/kernel/btf/vmlinux
```

Engineering Principle: Mounting `/sys/fs/cgroup` allows Tetragon to read the unified hierarchy tracking resource boundaries. When a process system call occurs inside a container, Tetragon intercepts the call at the kernel layer and references the active cgroup path to resolve exactly which container identity generated the event stream.

4.1.2 Exploit Execution via Absolute Node.js Path

As analyzed during environment reconnaissance, the target microservice utilizes a hardened base image that lacks traditional global interpreter shortcuts (/bin/sh or /bin/bash unmapped in standard pathways). To bypass this structural barrier and trigger the architectural alert baseline, the attack vector targets the native absolute runtime application binary directly:

```
ruben@kernel:~$ docker exec -it tetragon tetra getevents | grep vulnerable-app
```

Here, we have successful command execute which the objective is send a console.log for detect the threat for tetragon.

```
ruben@kernel:~$ sudo docker exec -it vulnerable-app /nodejs/bin/node -e "console.log('Alerta Tetragon')"
```

Alerta Tetragon
ruben@kernel:~\$

```
ruben@kernel:~$ docker exec -it tetragon tetra getevents | grep vulnerable-app
{"process_exec":{"process":{"exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","pid":5523,"uid":0,"cwd":"/home/ruben","binary":"/usr/bin/sudo","arguments":"docker exec -it vulnerable-app /nodejs/bin/node -e \"console.log('Alerta Tetragon')\"","flags":"execve clone","start_time":"2026-07-09T15:17:36.429323079Z","audid":1000,"parent_exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","tid":5522},"parent":{"exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","pid":5490,"uid":1000,"cwd":"/home/ruben","binary":"/bin/bash","flags":"execve clone","start_time":"2026-07-09T15:17:28.498875547Z","audid":1000,"parent_exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","tid":5490},"node_name":"137139056e09","time":"2026-07-09T15:17:36.429323019Z"}
{"process_exec":{"process":{"exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","pid":5523,"uid":0,"cwd":"/home/ruben","binary":"/usr/sbin/unix_chkpwd","arguments":"ruben chkexpiry","flags":"execve clone","start_time":"2026-07-09T15:17:36.434811198Z","audid":1000,"parent_exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","tid":5523},"parent":{"exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","pid":5522,"uid":0,"cwd":"/home/ruben","binary":"/usr/bin/sudo","arguments":"docker exec -it vulnerable-app /nodejs/bin/node -e \"console.log('Alerta Tetragon')\"","flags":"execve clone","start_time":"2026-07-09T15:17:36.429323079Z","audid":1000,"parent_exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","tid":5522},"node_name":"137139056e09","time":"2026-07-09T15:17:36.434810928Z"}
{"process_exit":{"process":{"exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","pid":5523,"uid":0,"cwd":"/home/ruben","binary":"/usr/sbin/unix_chkpwd","arguments":"ruben chkexpiry","flags":"execve clone","start_time":"2026-07-09T15:17:36.434811198Z","audid":1000,"parent_exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","tid":5523},"parent":{"exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","pid":5522,"uid":0,"cwd":"/home/ruben","binary":"/usr/bin/sudo","arguments":"docker exec -it vulnerable-app /nodejs/bin/node -e \"console.log('Alerta Tetragon')\"","flags":"execve clone","start_time":"2026-07-09T15:17:36.429323079Z","audid":1000,"parent_exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","tid":5522},"time":"2026-07-09T15:17:36.436350834Z","node_name":"137139056e09","time":"2026-07-09T15:17:36.436350834Z"}
{"process_exec":{"process":{"exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","pid":5525,"uid":0,"cwd":"/home/ruben","binary":"/usr/bin/docker","arguments":"exec -it vulnerable-app /nodejs/bin/node -e \"console.log('Alerta Tetragon')\"","flags":"execve clone","start_time":"2026-07-09T15:17:36.439265884Z","audid":1000,"parent_exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","tid":5525},"parent":{"exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","pid":5524,"uid":0,"cwd":"/home/ruben","binary":"/usr/bin/sudo","arguments":"docker exec -it vulnerable-app /nodejs/bin/node -e \"console.log('Alerta Tetragon')\"","flags":"execve","start_time":"2026-07-09T15:17:36.438044519Z","audid":1000,"parent_exec_id":"MTM3MTMSMDU2ZTA5OjE3Njg0NzU4NjQ5OTQ6NTUyMg==","refcnt":1,"tid":5524},"node_name":"137139056e09","time":"2026-07-09T15:17:36.439265774Z"}}
```

Step 5. Static Vulnerability Analysis with Trivy.

To establish a mature, enterprise-grade DevSecOps lifecycle, reactive runtime observability must be tightly coupled with proactive prevention controls. This section details the integration of static analysis pipelines executed against container images prior to production deployment, adhering to the "Shift Left" security methodology by identifying software vulnerabilities early in the delivery pipeline.

5.1 Image Vulnerability Scanning & Software Composition Analysis (SCA)

While kernel instrumentation safeguards active environments, **Trivy** is introduced as an advanced Software Composition Analysis (SCA) and Static Application Security Testing (SAST) engine. Trivy parses container filesystems, operating system package managers (e.g., Alpine, Debian metadata), and application-level dependency manifests (such as Node.js package.json.lock or Python requirements files) against updated global Common Vulnerabilities and Exposures (CVE) databases.

Once provisioned, a predictive analysis scan is triggered directly against the deployment candidate image from the local workspace directory:

```
sudo apt-get install wget apt-transport-https gnupg lsb-release -y
```

```
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | gpg --dearmor | sudo tee /usr/share/keyrings/trivy.gpg > /dev/null
```

```
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" | sudo tee /etc/apt/sources.list.d/trivy.list
```

```
sudo apt-get update && sudo apt-get install trivy -y
```

Then, we can scan an image of container, in that case the Owasp container, later some minutes, we will have the next information about vulnerabilities.

```
usuario@usuario-VMware-Virtual-Platform:~/server/ebpf-security$ trivy image bkimminich /juice-shop:v16.0.0
2026-07-09T17:39:01+02:00      INFO      [vulndb] Need to update DB
2026-07-09T17:39:01+02:00      INFO      [vulndb] Downloading vulnerability DB...
2026-07-09T17:39:01+02:00      INFO      [vulndb] Downloading artifact...      repo="
mirror.gcr.io/aquasec/trivy-db:2"
99.54 MiB / 99.54 MiB [-----] 100.00% 8.39 MiB p/s 12s
2026-07-09T17:39:14+02:00      INFO      [vulndb] Artifact successfully downloaded r
epo="mirror.gcr.io/aquasec/trivy-db:2"
2026-07-09T17:39:14+02:00      INFO      [vuln] Vulnerability scanning is enabled
2026-07-09T17:39:14+02:00      INFO      [secret] Secret scanning is enabled
2026-07-09T17:39:14+02:00      INFO      [secret] If your scanning is slow, please try
'--scanners vuln' to disable secret scanning
2026-07-09T17:39:14+02:00      INFO      [secret] Please see https://trivy.dev/docs/v0.
71/guide/scanner/secret#recommendation for faster secret detection
```


Here for example, a High vulnerability is about the private key who use the own container, is not safe.

```

/juice-shop/build/lib/insecurity.js (secrets)
Total: 1 (UNKNOWN: 0, LOW: 0, MEDIUM: 0, HIGH: 1, CRITICAL: 0)
HIGH: AsymmetricPrivateKey (private-key)
Asymmetric Private Key
/juice-shop/build/lib/insecurity.js:47 (offset: 2805 bytes) (added by 'COPY /juice-shop . # buildkit')
45  const z85 = _importStar(require("z85"));
46  exports.publicKey = fs_1.default ? fs_1.default.readFileSync('encryptionkeys/jwt.pub', 'utf8') : 'pl
47  [ ----BEGIN RSA PRIVATE KEY-----*****
*****
*****
*****-----END RSA PRIVATE
48  const hash = (data) => crypto_1.default.createHash('md5').update(data).digest('hex');
```

Futhermore, trivy give a report on the begining about the image of container.

```

usuario@usuario-Vmware-Virtual-Platform:~/server/bf-security$ trivy image bkimminich/juice-shop:v16.0.0
2026-07-09T17:40:57+02:00 INFO [vuln] Vulnerability scanning is enabled
2026-07-09T17:40:57+02:00 INFO [secret] Secret scanning is enabled
2026-07-09T17:40:57+02:00 INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-07-09T17:40:57+02:00 INFO [secret] Please see https://trivy.dev/docs/v0.71/guide/scanner/secret#recommendation for faster secret detection
2026-07-09T17:40:58+02:00 INFO Detected OS family='debian' version='11.8'
2026-07-09T17:40:58+02:00 INFO [debian] Detecting vulnerabilities... os_version="11" pkg_name=9
2026-07-09T17:40:58+02:00 INFO [num] Number of language-specific files num=1
2026-07-09T17:40:58+02:00 INFO [node-pkg] Detecting vulnerabilities...
2026-07-09T17:40:58+02:00 WARN Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/docs/v0.71/guide/scanner/vulnerability#severity-selection for details.
2026-07-09T17:40:58+02:00 INFO Table result includes only package filenames. Use '--format json' option to get the full path to the package file.

```

Target	Type	Vulnerabilities	Secrets
bkminich/juice-shop:v16.0.0 (debian 11.8)	debian	83	-
juice-shop/build/package.json	node-pkg	0	-
juice-shop/frontend/package.json	node-pkg	0	-
juice-shop/node_modules/@aashutoshrathi/word-wrap/package.json	node-pkg	0	-
juice-shop/node_modules/@adraffy/ens-normalize/package.json	node-pkg	0	-
juice-shop/node_modules/@ampproject/remapping/package.json	node-pkg	0	-
juice-shop/node_modules/@babel/code-frame/package.json	node-pkg	0	-
juice-shop/node_modules/@babel/compat-data/package.json	node-pkg	0	-
juice-shop/node_modules/@babel/core/node_modules/convert-source-map/package.json	node-pkg	0	-
juice-shop/node_modules/@babel/core/node_modules/semver/package.json	node-pkg	0	-
juice-shop/node_modules/@babel/core/package.json	node-pkg	1	-

5.1.2 Static Metric Assessment & Vulnerability Vectors

The localized report summary outputs critical structural posture details:

1. **Base OS Attack Surface:** Trivy isolates exactly eighty-three (83) known vulnerabilities within the foundational operating system layer (debian 11.8), emphasizing why unpatched core images pose an entry mechanism for sandbox escapes.
2. **Application-Level Manifest Breakdown:** The engine audits the application package tree, targeting transitive libraries. It maps flaws directly down to precise internal configurations, flagging issues in deep application modules like @babel/core (1 vulnerability), form-data (2 vulnerabilities), and braces (1 vulnerability).
3. **Secret Leakage Boundaries:** The report lists a clean indicator across localized dependency objects (Secrets: -), verifying that structural application code elements are clear of exposed cryptographic identifiers prior to instantiation.

CONCLUSION & Key Takeaways

The implementation of this hybrid security infrastructure demonstrates the synergy between proactive and reactive defense lines under the DevSecOps model.

By embedding **Trivy** in the management ecosystem, the architecture blocks vulnerable images and credentials early in the engineering lifecycle (*Shift-Left Prevention*).

Concurrently, **Cilium Tetragon** addresses the inevitability of post-exploitation events by monitoring low-level kernel process contexts (`sys_enter_execve`) via non-intrusive eBPF tracing. This layered topology ensures absolute visibility across both static deployment states and live container runtimes without degrading application performance.